FRENCH UNIVERSITY IN ARMENIA

Faculty of Computer science and Applied mathematics

Subject **Databases 1**

Students **Artyom MKRTCHYAN | Mane MAZMANDYAN | Davit ARAKELYAN**

Teacher **Varazdat AVETISYAN**

April 2026

Flight Monitoring System

Contents

Team Contribution **4**

Problem Description and Requirements Analysis **5**

Conceptual Design Explanation **5**

- Main Data Objects 6
- Main Business Rules 6
- System Data Requirements 6
- System Processes **Error! Bookmark not defined.**
- Flow Charts 7

ER Diagram and Logical Schema description **9**

- ER Model Transformation 11
- Relational Schema 12
- Relationships and Cardinalities 12
- Constraints 13
- Design Decisions 13

Normalization Process Documentation **14**

- First Normal Form (1NF) 14
- Second Normal Form (2NF) 14
- Third Normal Form (3NF) 14

Relational Algebra Query Demonstrations **15**

SQL Scripts **17**

Documentation of Views, Indexes, Triggers, and Stored Procedures **18**

Database Deployment Description **20**

Project Conclusions and Results **21**

Team Contribution

The Flight Management System project involved collaborative efforts across multiple areas of database design, implementation, and integration. This table highlights the areas of responsibility, allowing each team member to be credited for their contributions in designing, implementing, and optimizing the database system, as well as integrating it with the user interface and documenting the project. The team contributions are summarized below:

Project Area	Team Member(s)
Project Planning & Requirements Analysis	Worked Together
Conceptual Design (ER Diagram, Entities & Relationships)	Mane Mazmandyan
Logical Design (Relational Schema, Normalization)	David Arakelyan
Physical Database Implementation	Mane Mazmandyan
Data Population & DML	David Arakelyan
Views, Indexes, Triggers, Stored Procedures	Artyom Mkrtchyan
Database Testing & Query Optimization	David Arakelyan
Frontend/UI Development (React interface, user interactions)	Artyom Mkrtchyan
Final Report & Presentation	Worked Together

Problem Description and Requirements Analysis

The project is dedicated to the design, development, and implementation of a comprehensive relational database system intended to support and optimize airline operations management. The primary objective of this system is to efficiently store, organize, and manage large volumes of structured data related to various aspects of airline activity, including airlines, aircraft, airports, routes, flights, passengers, and ticketing processes.

The database architecture is based on a set of core entities, each represented by a dedicated table. These include **Aircraft**, **Airlines**, **Airports**, **Cities**, **Countries**, **Flights**, **Passengers**, **Routes**, **Tickets**, and **TicketAuditLog**. Each table is carefully designed with appropriate attributes, primary keys, and constraints to ensure data accuracy and eliminate redundancy. The relationships between these entities are established through foreign keys, enabling consistent linkage across the system. For instance, each aircraft is associated with a specific airline, each airport is located within a city and a country, and each route defines a valid connection between two airports.

A key focus of the project is maintaining data integrity and consistency across all interconnected components. This is achieved through the implementation of relational constraints such as primary keys, foreign keys, unique constraints, and check conditions. Additionally, normalization principles (up to Third Normal Form) are applied to reduce data duplication and improve database efficiency. For example, separating cities and countries into distinct tables ensures that geographic data is not redundantly stored across multiple records.

The system also addresses complex operational scenarios. Each flight is linked to both a route and an aircraft, ensuring that only valid combinations are allowed. The ticketing subsystem connects passengers to specific flights and includes attributes such as seat number, class, and booking details. To enhance traceability and accountability, a **TicketAuditLog** table is implemented, which records all changes made to ticket records, including insertions, updates, and deletions. This allows for tracking historical modifications and supports auditing requirements.

From a functional perspective, the database supports full CRUD (Create, Read, Update, Delete) operations for all entities. It also includes advanced SQL features such as stored procedures, triggers, and views to automate routine processes and enforce business logic. For example, triggers can be used to automatically assign seats or log ticket changes, while stored procedures handle complex operations like booking a flight or validating passenger information.

In addition to operational functionality, the system is designed to support analytical queries. These may include retrieving flight schedules, analyzing airline performance, tracking passenger activity, or identifying underutilized routes. Such queries are essential for decision-making and operational optimization.

The database is built with scalability and extensibility in mind. Its modular structure allows for easy integration with external systems such as web applications, booking platforms, or reporting tools. Future enhancements could include additional features like pricing models, loyalty programs, or real-time flight tracking.

Overall, this project delivers a robust and well-structured relational database solution for managing airline operations. It ensures high levels of data consistency, integrity, and performance while providing flexibility for future expansion and integration.

Conceptual Design Explanation

The conceptual design of the system is based on the analysis of the airline management domain and its core operations. The system models the real-world entities involved in airline operations and defines the relationships between them.

- Main Data Objects

  - **Airlines** – represents airline companies
  - **Aircraft** – represents airplanes belonging to airlines
  - **Airports** – represents departure and arrival locations
  - **Cities** and **Countries** – represent geographical hierarchy
  - **Routes** – define connections between airports
  - **Flights** – represent scheduled flights
  - **Passengers** – store passenger information
  - **Tickets** – represent bookings made by passengers

- Main Business Rules

Airline Operations:

  - Each airline can operate multiple aircraft, but no aircraft can belong to more than one airline at a time.
  - Airlines must have at least one active route to operate flights.

Aircraft Availability

  - An aircraft can operate only one flight at any given time.
  - Aircraft maintenance records must be up-to-date before scheduling a flight.

Flight Scheduling

  - Each flight must have a scheduled departure and arrival time.
  - Flights cannot be scheduled if the assigned aircraft is unavailable or under maintenance.
  - Flights can have multiple classes of tickets (e.g., economy, business), and the number of seats sold must not exceed the aircraft's capacity.

Passenger Management

  - Each passenger must have a unique identifier (passport or ID number) to book tickets.
  - Passengers can book multiple tickets but cannot be assigned to overlapping flights at the same time.

Ticketing Rules

  - Tickets must specify the class and seat number.
  - Cancellations or changes must update the available seat count for the flight.

Route and Airport Constraints

- Routes must always connect two different airports.
- A route can have multiple flights, but each flight is uniquely identified by flight number and date.

Pricing and Revenue Rules

- Ticket prices must be set according to route distance.

System Data Requirements

- **Entity Identifiers**: unique IDs for Aircraft, Airline, Flight, Route, Passenger, Ticket.
- **Relationships**: foreign keys link aircraft → airline, flight → aircraft & route, ticket → passenger & flight.
- **Operational Data**: flight dates, times, statuses; route airports; aircraft availability & maintenance.
- **Passenger & Booking Details**: name, passport, contact; ticket class, seat, booking date, payment; optional preferences.

System Processes

- **Manage Airlines & Aircraft**: register, assign, track status.
- **Manage Routes**: create routes connecting valid airports.
- **Schedule Flights**: assign aircraft to routes, check availability & maintenance.
- **Book Tickets**: link passengers to flights, assign seats, update availability.
- **Data Retrieval**: queries & views for reports, monitoring, and decision-making.

- Flow Charts

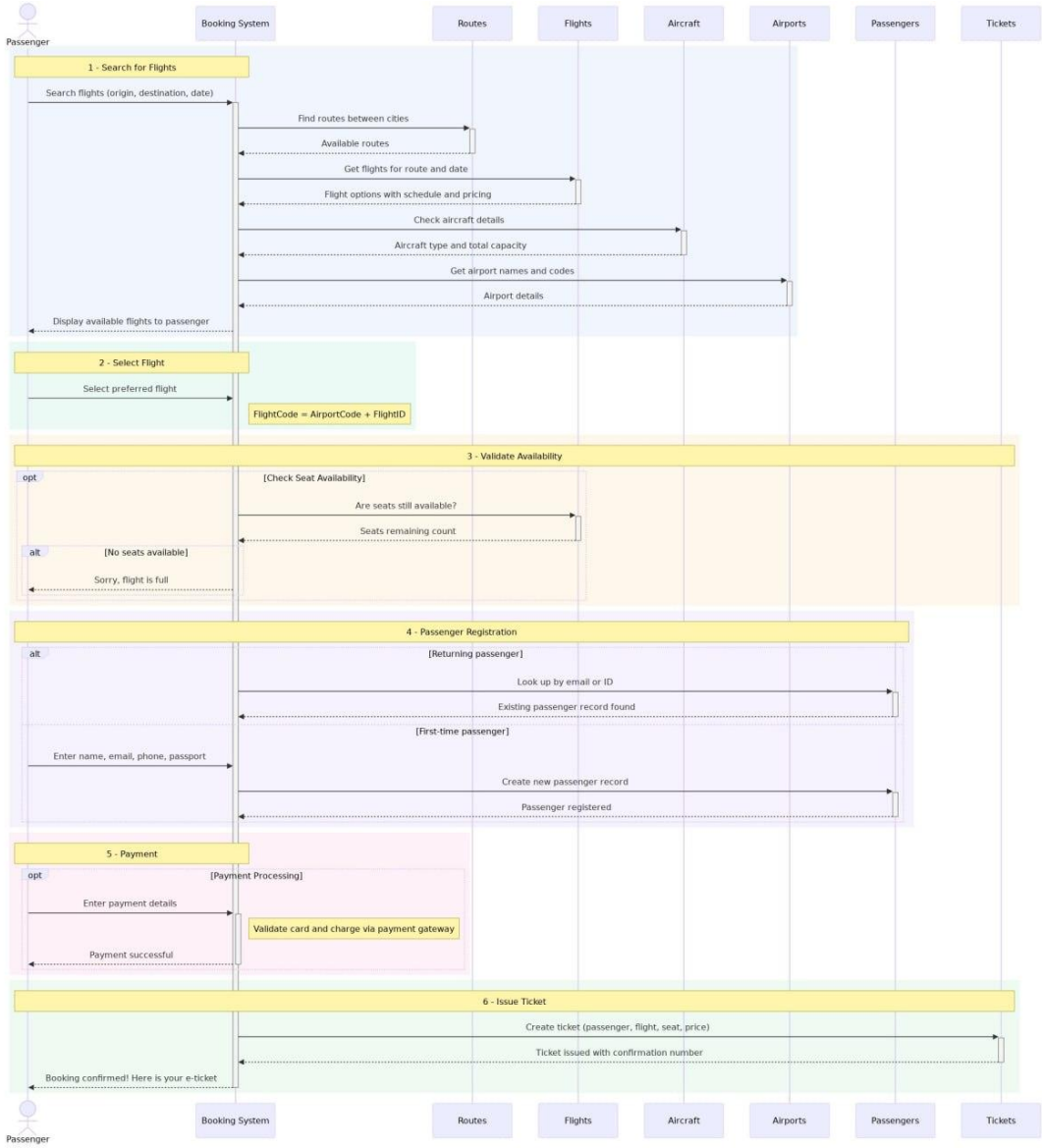


Figure 1: Sequence Diagram

The sequence diagrams illustrate the step-by-step interactions between system components during key processes such as flight booking, ticket cancellation, and flight information retrieval. They show how messages are exchanged between actors (such as passengers or users) and system objects (such as Flights, Tickets, and the Database), following a clear chronological order. This helps to understand how each request is processed from the moment it is initiated until the final result is returned.

For example, during flight booking, the system validates user input, checks availability, and creates a ticket in the database. In ticket cancellation, it verifies and updates or removes the ticket, while in flight information retrieval, it processes queries and returns relevant data. Overall, sequence diagrams help visualize system behavior, ensure correct interaction between components, and make it easier to identify and prevent potential errors.

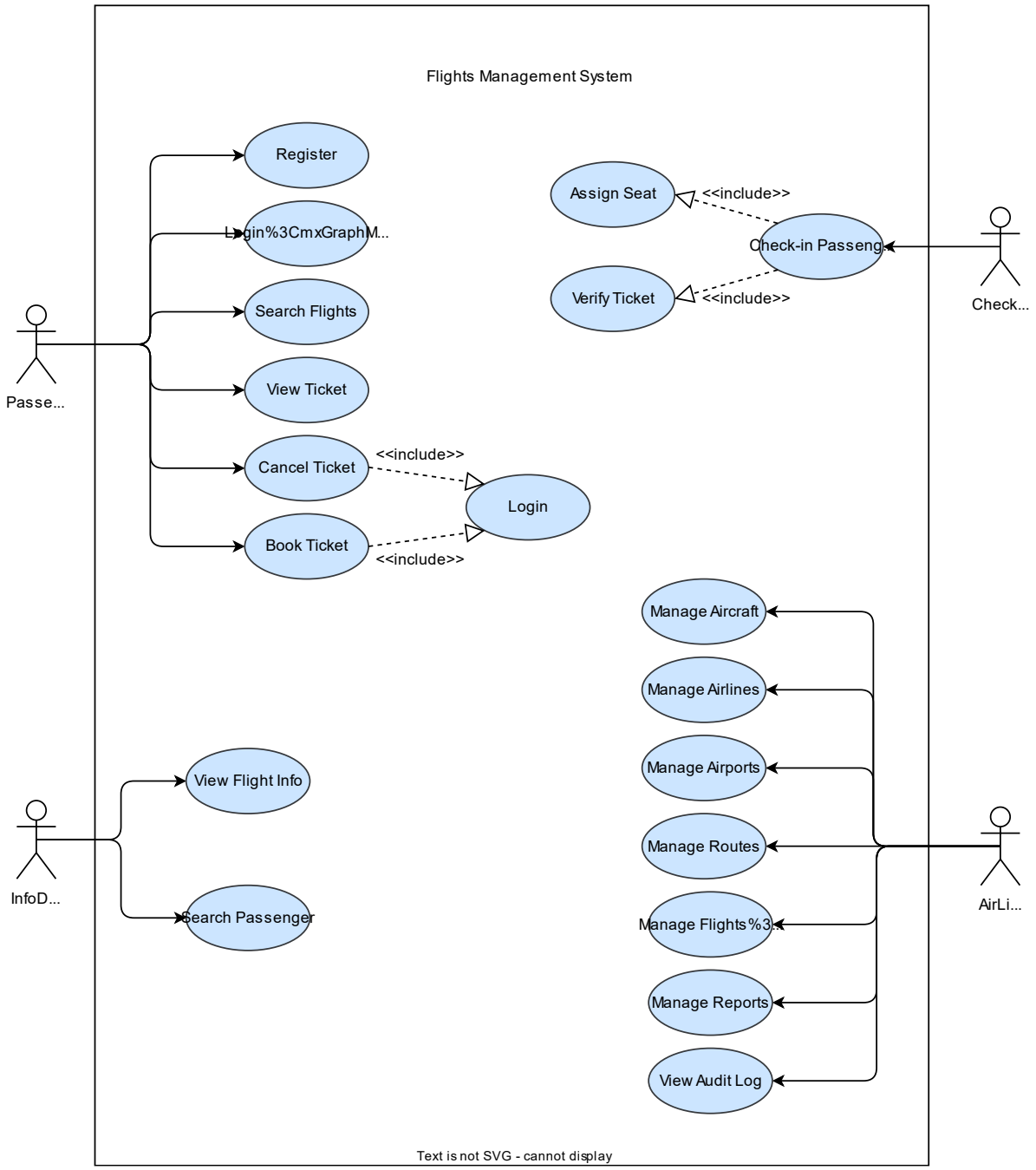


Figure 2: Use-case Diagram

The use case diagrams provide a high-level overview of the system's functionalities, showing how different users interact with the system. They present the system from the user's perspective, focusing on what actions can be performed rather than how they are implemented. Each use case represents a specific goal, such as booking a ticket, searching for flights, or cancelling a reservation, and clearly identifies the actors involved and their roles in the process.

These diagrams help define the scope of the system by outlining all major functionalities and the relationships between actors and use cases. They also make it easier to understand user requirements, ensure that all key operations are covered, and provide a simple and clear visualization for both developers and stakeholders. Overall, use case diagrams serve as an essential tool for planning, communication, and system analysis.

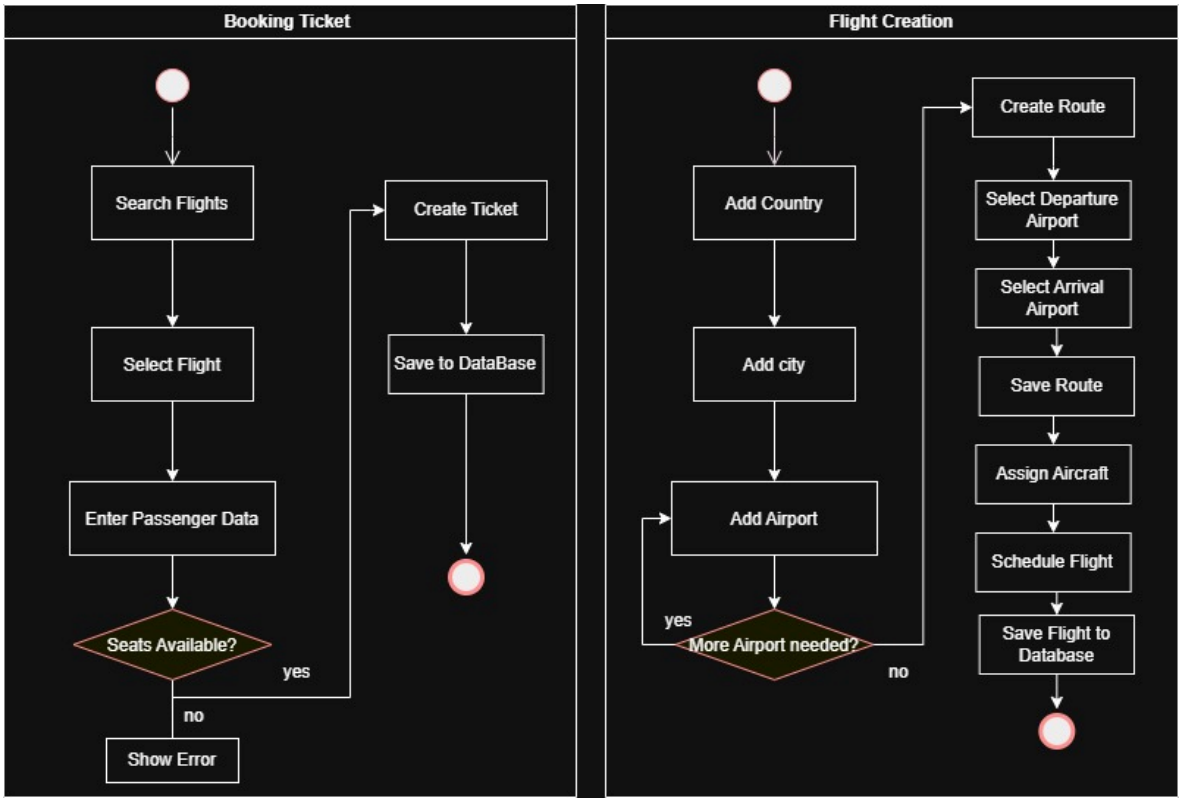


Figure 3: Activity Diagram

The activity diagrams capture the workflow of major processes, highlighting the sequence of tasks, decision points, and possible parallel actions. They provide a clear representation of how different activities are performed within the system, showing the flow from the initial step to the final outcome. This makes it easier to understand how processes are structured and how different steps are connected.

These diagrams are especially useful for modeling complex operations, as they help identify alternative paths, conditions, and dependencies between actions. By visualizing the system's dynamic behavior, activity diagrams allow developers and stakeholders to verify that all processes are logically consistent, complete, and efficiently organized.

ER Diagram and Logical Schema description

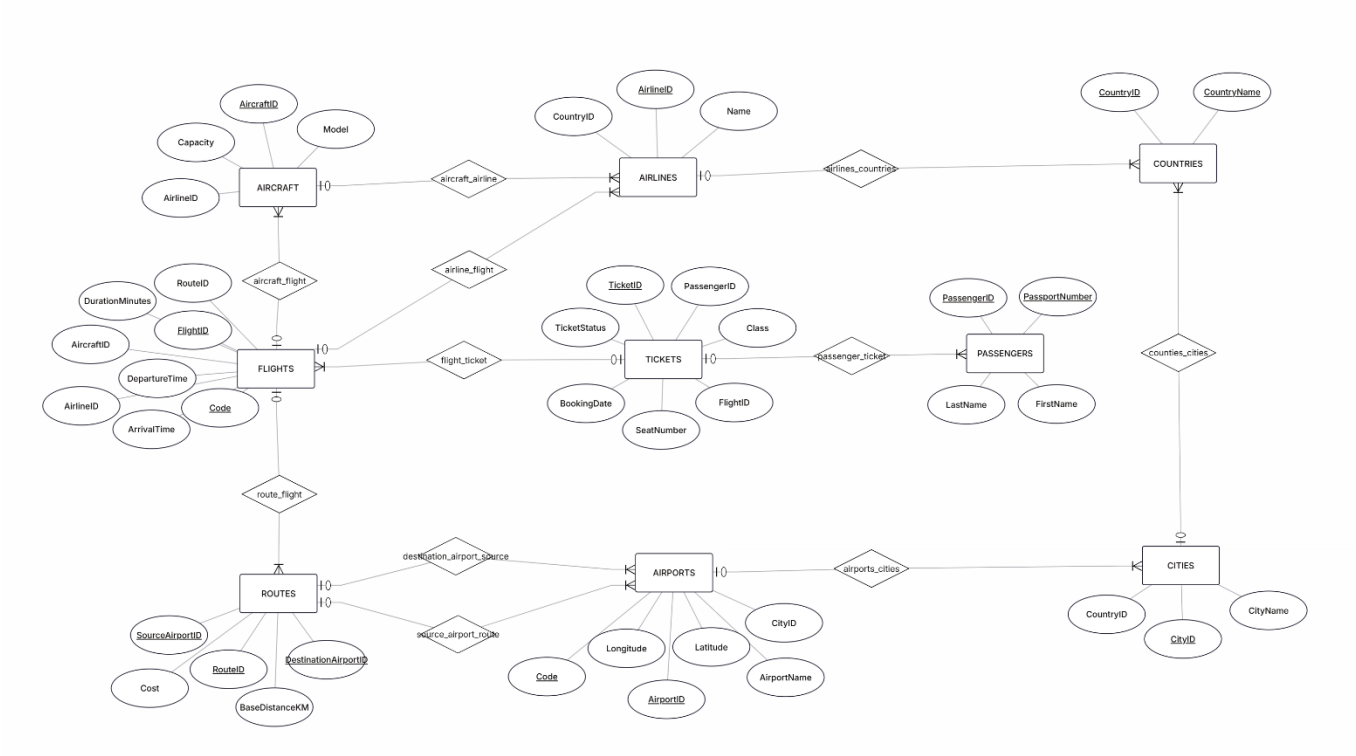


Figure 4 ER Diagram

- ER Model Transformation

The Entity–Relationship (ER) model was transformed into a relational schema to enable implementation within a relational database management system. Each entity from the ER diagram is represented as a separate table, with its attributes converted into columns and a primary key defined to uniquely identify each record. This transformation ensures that the conceptual design is properly translated into a structured and organized database format.

Relationships between entities are implemented using foreign keys, which establish links between related tables and maintain referential integrity. This ensures that all references between records remain valid, preventing inconsistencies such as orphaned data. As a result, the relational schema preserves the logic of the original ER model while supporting efficient data storage, retrieval, and management.

- Relational Schema

Aircraft	
AircraftID	int
Model	Varchar(100)
Capacity	int
AirlineID	int

Airlines	
AirlineID	int
Name	varchar(150)
CountryID	int

Airports	
AirportID	int
Code	char(3)
AirportName	varchar(150)
CityID	int
Latitude	decimal(9,6)
Longitude	decimal(9,6)

Cities	
CityID	int
CityName	nvarchar(100)
CountryID	int

Countries	
CountryId	int
CountryName	Varchar(100)

Flights	
FlightID	int
RouteID	int
AirlineID	int
AircraftID	int
DepartureTime	datetime
ArrivalTime	datetime
DurationMinutes	int
FlightCode	varchar(20)

Passengers	
PassengerID	int
FirstName	varchar(100)
LastName	varchar(100)
PassportNumber	varchar(50)

Routes	
RouteID	int
SourceAirportID	int
DestinationAirportID	int
BaseDistanceKM	int
Cost	decimal(10,2)

Tickets	
TicketID	int
PassengerID	int
FlightID	int
SeatNumber	varchar(5)
BookingDate	date
TicketStatus	varchar(50)
Class	nvarchar(20)

- ## Relationships and Cardinalities

  - One airline can own many aircraft (1:N)
  - One airline can operate many flights (1:N)
  - One country can contain many cities (1:N)
  - One city can contain many airports (1:N)
  - One airport can be used in many routes (1:N)
  - One route connects two airports source and destination (1:N)
  - One route can have many flights (1:N)
  - One aircraft can be assigned to many flights (1:N)
  - One passenger can have many tickets (1:N)
  - One flight can have many tickets (1:N)
  - One country can have many airlines (1:N)

- ## Constraints

  - Primary keys uniquely identify each record and cannot be NULL
  - Foreign keys ensure referential integrity between related tables
  - Each aircraft must belong to an existing airline
  - Each flight must reference a valid route, airline, and aircraft
  - Each route must reference valid source and destination airports
  - Each ticket must reference a valid passenger and flight
  - PassportNumber should be unique for each passenger

- ## Design Decisions

  - The separation of Countries, Cities, and Airports ensures normalization and avoids redundancy
  - Routes are modeled as a separate entity to represent connections between airports
  - Flights are linked to both routes and aircraft to ensure consistency in scheduling
  - Tickets act as a bridge between passengers and flights
  - Additional attributes such as DurationMinutes and Cost are included for analytical queries

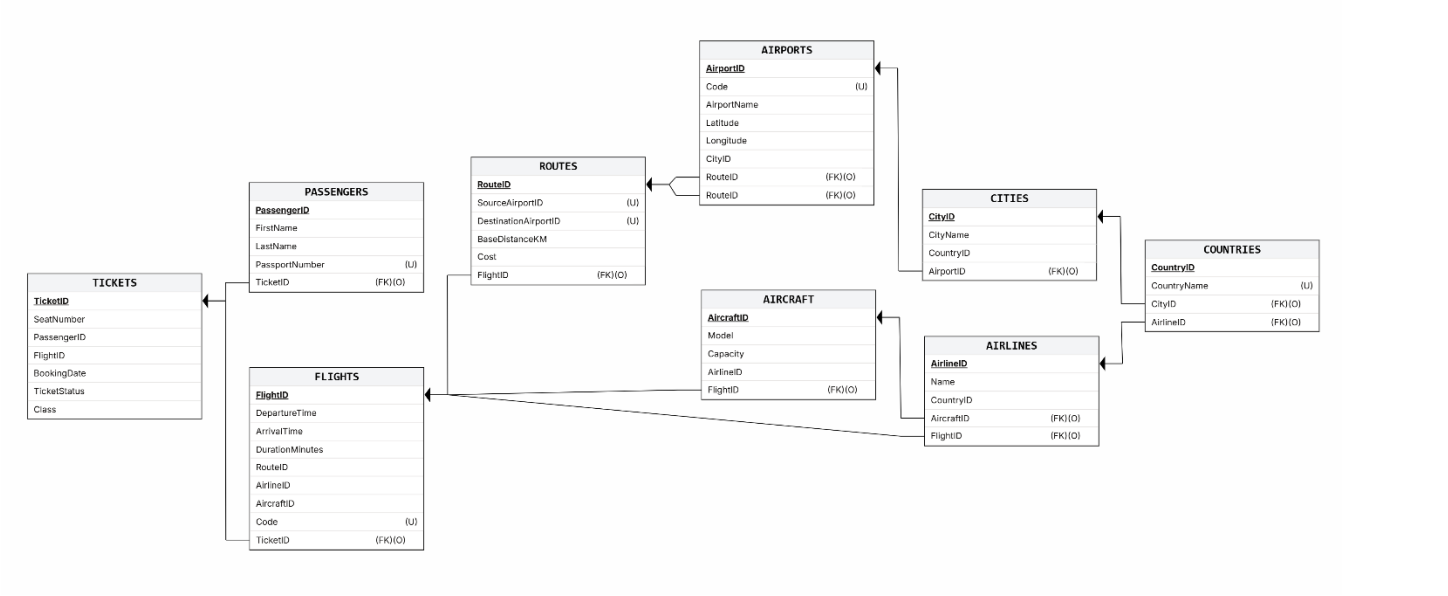


Figure 5: Relationships Diagram

Normalization Process Documentation

- ## First Normal Form (1NF)

The database satisfies the First Normal Form (1NF) as:

- All attributes contain atomic (indivisible) values
- There are no repeating groups or multi-valued attributes
- Each table has a defined primary key

For example, in the **Passengers** table, attributes such as FirstName, LastName, and PassportNumber store single values, and each record is uniquely identified by PassengerID.

- ## Second Normal Form (2NF)

The database satisfies the Second Normal Form (2NF) because:

- It is already in 1NF
- All non-key attributes are fully functionally dependent on the primary key

All tables use single-attribute primary keys (e.g., FlightID, TicketID), so partial dependency does not exist.

For example:

- In the **Flights** table, attributes such as DepartureTime and ArrivalTime depend entirely on FlightID
- In the **Tickets** table, SeatNumber and BookingDate depend only on TicketID

- ## Third Normal Form (3NF)

The database satisfies the Third Normal Form (3NF) because:

- It is already in 2NF
- There are no transitive dependencies

Non-key attributes do not depend on other non-key attributes.

For example:

- Country information is stored in the **Countries** table, not in Cities or Airports
- City information is stored in the **Cities** table, avoiding duplication in Airports
- Airline information is separated from Aircraft

This eliminates redundancy and ensures data consistency.

Relational Algebra Query Demonstrations

At this stage, the work has been carried out at a conceptual level, demonstrating how relational algebra operations are applied to the database schema. These operations include selection, projection, join, union, intersection, and difference, each representing a fundamental mechanism for querying and manipulating data within a relational model. They help illustrate how information can be filtered, combined, and transformed based on specific conditions.

By applying these operations to the schema, it becomes possible to model typical data retrieval scenarios and verify the correctness of relationships between tables. This step also provides a theoretical foundation for later implementation in SQL, ensuring that queries are logically sound and aligned with the structure of the database.

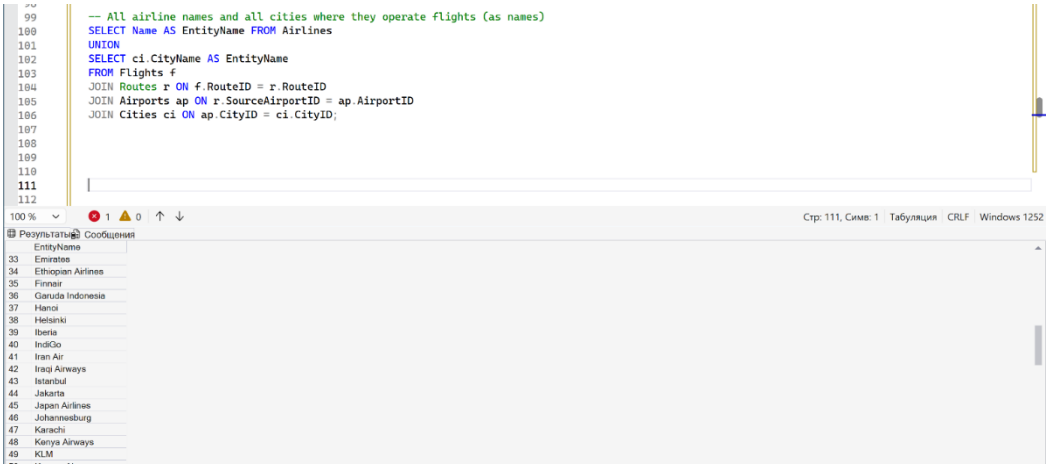


Figure 6: Union

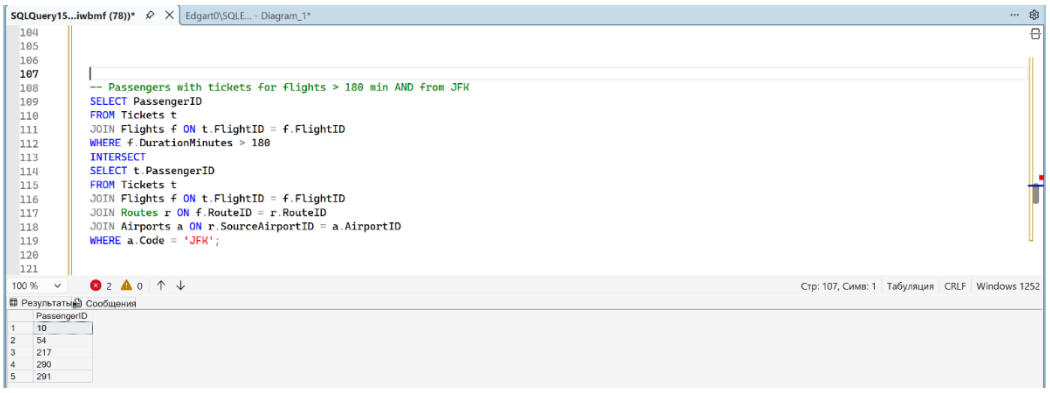


Figure 7: Intersection

SQLQuery15...iwbmf (78))* Edgart0\SQLE... - Diagram_1*

```sql
-- Flights with departure and arrival airport info
SELECT f.*, r.*, ap_src.AirportName AS SourceAirport, ap_dst.AirportName AS DestinationAirport
FROM Flights f
JOIN Routes r ON f.RouteID = r.RouteID
JOIN Airports ap_src ON r.SourceAirportID = ap_src.AirportID
JOIN Airports ap_dst ON r.DestinationAirportID = ap_dst.AirportID;
```

100 % 1 0 Стр: 67, Симв: 1 Табуляция CRLF Windows 1252

Результаты Сообщения

	FlightID	RouteID	AirlineID	AircraftID	DepartureTime	ArrivalTime	DurationMinutes	FlightCode	RouteID	SourceAirportID	DestinationAirportID	BaseDistanceKM	Cost	SourceAirport	Destin
1	5	394	162	1106	2026-03-20 08:30:00.000	2026-03-20 10:45:00.000	135	UZJ435	394	479	484	5540	831.00	John F. Kennedy International Airport	Londo
2	105	394	192	1206	2026-03-20 07:45:00.000	2026-03-20 09:55:00.000	130	RES348	394	479	484	5540	831.00	John F. Kennedy International Airport	Londo
3	113	394	202	1214	2026-03-22 14:10:00.000	2026-03-22 16:25:00.000	135	JEB330	394	479	484	5540	831.00	John F. Kennedy International Airport	Londo
4	149	394	201	1148	2026-03-28 14:00:00.000	2026-03-28 16:15:00.000	135	ATP353	394	479	484	5540	831.00	John F. Kennedy International Airport	Londo
5	6	395	168	1107	2026-03-20 14:15:00.000	2026-03-20 17:00:00.000	165	OOG401	395	479	486	5845	876.00	John F. Kennedy International Airport	Charle
6	137	395	200	1146	2026-03-24 05:45:00.000	2026-03-24 08:15:00.000	150	LAC084	395	479	486	5845	876.00	John F. Kennedy International Airport	Charle
7	7	396	162	1108	2026-03-20 20:00:00.000	2026-03-20 22:10:00.000	130	XPZ373	396	479	482	570	86.00	John F. Kennedy International Airport	Toront
8	8	397	163	1109	2026-03-21 06:45:00.000	2026-03-21 09:15:00.000	150	QNH123	397	484	524	5500	825.00	London Heathrow Airport	Dubai
9	9	398	169	1110	2026-03-21 11:30:00.000	2026-03-21 14:40:00.000	190	YEQ282	398	484	487	1430	215.00	London Heathrow Airport	Leona
10	141	398	167	1121	2026-03-28 09:15:00.000	2026-03-28 11:45:00.000	150	XQM159	398	484	487	1430	215.00	London Heathrow Airport	Leona
11	10	399	163	1111	2026-03-21 16:00:00.000	2026-03-21 17:50:00.000	110	RLH346	399	484	511	2460	369.00	London Heathrow Airport	Athens
12	11	400	164	1112	2026-03-21 21:10:00.000	2026-03-22 00:20:00.000	190	FCQ578	400	486	507	430	65.00	Charles de Gaulle Airport	Amste
13	12	401	164	1113	2026-03-22 07:40:00.000	2026-03-22 10:20:00.000	160	THQ011	401	486	485	1050	158.00	Charles de Gaulle Airport	Berlin
14	13	402	187	1114	2026-03-22 13:20:00.000	2026-03-22 17:50:00.000	270	NPR640	402	486	524	5240	786.00	Charles de Gaulle Airport	Dubai
15	14	403	165	1115	2026-03-22 19:15:00.000	2026-03-22 21:45:00.000	150	YNY912	403	481	494	1710	256.00	Sheremetyevo International Airport	Istanbu
16	15	404	168	1116	2026-03-23 05:00:00.000	2026-03-23 07:45:00.000	165	LLQ222	404	481	480	5790	869.00	Sheremetyevo International Airport	Beijing
17	16	405	166	1117	2026-03-23 09:30:00.000	2026-03-23 11:50:00.000	140	HXB303	405	481	485	1600	240.00	Sheremetyevo International Airport	Berlin

Figure 8: Join

SQLQuery15...iwbmf (78))* Edgart0\SQLE... - Diagram_1*

```sql
-- Tickets with status 'Booked'
SELECT * FROM Tickets
WHERE TicketStatus = 'Booked';
```

100 % 1 0 Стр: 17, Симв: 1 Табуляция CRLF Windows 1252

Результаты Сообщения

	TicketID	PassengerID	FlightID	SeatNumber	BookingDate	TicketStatus	Class
1	16	3	15	18F	2026-03-12	Booked	Economy
2	19	25	30	04A	2026-03-10	Booked	Business
3	21	40	40	11E	2026-03-15	Booked	Economy
4	23	52	50	25A	2026-03-12	Booked	Economy
5	26	81	65	30F	2026-03-18	Booked	Economy
6	30	170	85	21B	2026-03-16	Booked	Economy
7	32	175	95	14D	2026-03-13	Booked	Economy
8	33	180	100	05C	2026-03-21	Booked	Business
9	37	195	120	11D	2026-03-15	Booked	Economy
10	40	210	135	15F	2026-03-12	Booked	Economy
11	41	215	140	01E	2026-03-17	Booked	Business
12	43	228	150	07B	2026-03-09	Booked	Economy
13	45	240	6	31D	2026-03-11	Booked	Economy
14	47	255	16	03A	2026-03-05	Booked	Business
15	49	268	26	24E	2026-03-13	Booked	Economy
16	51	280	36	10F	2026-03-22	Booked	Economy
17	53	288	46	04E	2026-03-12	Booked	Business
18	56	305	61	02B	2026-03-14	Booked	Business

Figure 9: Selection

SQLQuery15...iwbmf (78))* Edgart0\SQLE... Diagram_1*

```sql
-- All passenger names
SELECT FirstName, LastName FROM Passengers;
```

100 % 1 0 Стр: 40, Симв: 1 Табуляция CRLF Windows 1252

Результаты Сообщения

	FirstName	LastName
1	John	Smith
2	Maria	Garcia
3	Li	Wei
4	Olga	Ivanova
5	Carlos	Santos
6	Emma	Brown
7	Sofia	Rossi
8	Pierre	Dubois
9	Anna	Müller
10	Ahmed	Ali
11	David	Lee
12	Isabella	Martinez
13	Tom	Wilson
14	Hana	Kim
15	Mustafa	Yilmaz
16	Sara	Johansson
17	Nikolai	Petrov
18	Layla	Hassan

Figure 10: Projection

SQL Scripts

```sql
CREATE TABLE Flights (
    FlightID INT PRIMARY KEY IDENTITY(1,1),
    AirlineID INT NOT NULL,
    RouteID INT NOT NULL,
    DepartureTime DATETIME NOT NULL,
    ArrivalTime DATETIME NOT NULL,
    FOREIGN KEY (AirlineID) REFERENCES Airlines(AirlineID),
    FOREIGN KEY (RouteID) REFERENCES Airports(AirportID)
);
```

Figure 11: DDL Statement

```sql
INSERT INTO Cities (CityName, CountryID)
VALUES
('New York', (SELECT CountryID FROM Countries WHERE CountryName='United States')),
('Beijing', (SELECT CountryID FROM Countries WHERE CountryName='China')),
('Moscow', (SELECT CountryID FROM Countries WHERE CountryName='Russia')),
('Toronto', (SELECT CountryID FROM Countries WHERE CountryName='Canada')),
('São Paulo', (SELECT CountryID FROM Countries WHERE CountryName='Brazil')),
('London', (SELECT CountryID FROM Countries WHERE CountryName='United Kingdom')),
('Berlin', (SELECT CountryID FROM Countries WHERE CountryName='Germany')),
('Paris', (SELECT CountryID FROM Countries WHERE CountryName='France')),
('Rome', (SELECT CountryID FROM Countries WHERE CountryName='Italy')),
('Madrid', (SELECT CountryID FROM Countries WHERE CountryName='Spain')),
('Mexico City', (SELECT CountryID FROM Countries WHERE CountryName='Mexico')),
('Tokyo', (SELECT CountryID FROM Countries WHERE CountryName='Japan')),
('New Delhi', (SELECT CountryID FROM Countries WHERE CountryName='India')),
('Sydney', (SELECT CountryID FROM Countries WHERE CountryName='Australia')),
('Seoul', (SELECT CountryID FROM Countries WHERE CountryName='South Korea')),
('Istanbul', (SELECT CountryID FROM Countries WHERE CountryName='Turkey')),
('Riyadh', (SELECT CountryID FROM Countries WHERE CountryName='Saudi Arabia')),
('Johannesburg', (SELECT CountryID FROM Countries WHERE CountryName='South Africa')),
('Cairo', (SELECT CountryID FROM Countries WHERE CountryName='Egypt')),
('Lagos', (SELECT CountryID FROM Countries WHERE CountryName='Nigeria')),
('Buenos Aires', (SELECT CountryID FROM Countries WHERE CountryName='Argentina')),
('Bogotá', (SELECT CountryID FROM Countries WHERE CountryName='Colombia')),
```

Figure 12: DML Statement

```sql
SELECT f.FlightID, a.Name AS AirlineName, f.DepartureTime, f.ArrivalTime
FROM Flights f
JOIN Airlines a ON f.AirlineID = a.AirlineID
WHERE a.Name = 'Air France';
```

Результаты | Сообщения

	FlightID	AirlineName	DepartureTime	ArrivalTime
1	9	Air France	2026-03-21 11:30:00.000	2026-03-21 14:40:00.000
2	23	Air France	2026-03-25 09:00:00.000	2026-03-25 12:20:00.000
3	31	Air France	2026-03-27 16:30:00.000	2026-03-27 19:40:00.000
4	60	Air France	2026-03-21 18:20:00.000	2026-03-21 21:50:00.000
5	76	Air France	2026-03-26 12:15:00.000	2026-03-26 15:35:00.000
6	77	Air France	2026-03-26 18:40:00.000	2026-03-26 21:40:00.000
7	88	Air France	2026-03-30 06:30:00.000	2026-03-30 09:20:00.000
8	106	Air France	2026-03-20 13:20:00.000	2026-03-20 16:10:00.000
9	130	Air France	2026-03-27 10:45:00.000	2026-03-27 13:20:00.000

Figure 13: DQL Statement

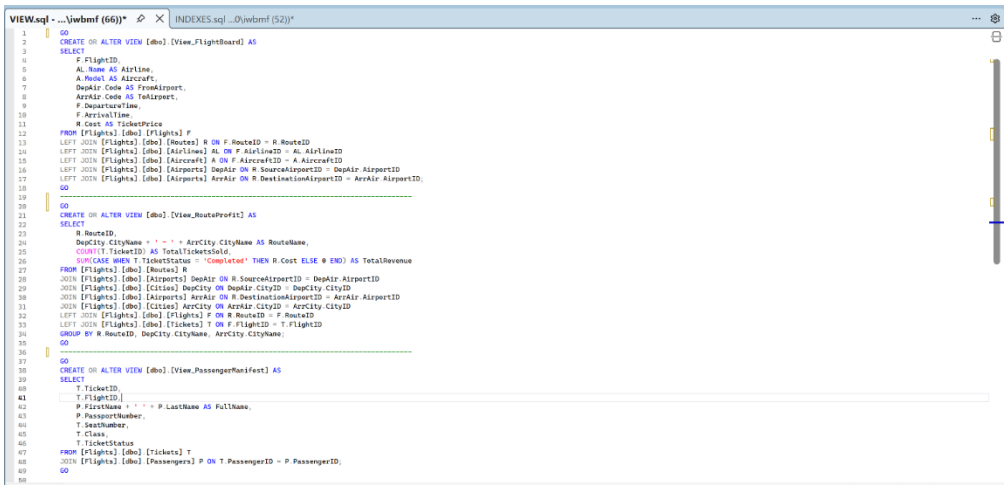
```sql
CREATE ROLE [InfoDesk_Agent1];
CREATE ROLE [Airport_Staff1];
CREATE ROLE [Airline_Internal_Staff1];

GRANT SELECT ON [dbo].[View_FlightBoard] TO [InfoDesk_Agent];
GRANT SELECT ON [dbo].[View_PassengerManifest] TO [InfoDesk_Agent];
GRANT EXECUTE ON [dbo].[sp_FindFlightsByCities] TO [InfoDesk_Agent];

GRANT SELECT ON SCHEMA::dbo TO [Airport_Staff];
GRANT UPDATE ON [dbo].[Tickets] TO [Airport_Staff];
GRANT UPDATE ON [dbo].[Passengers] TO [Airport_Staff];
DENY DELETE, INSERT ON [dbo].[Flights] TO [Airport_Staff];
DENY DELETE, INSERT ON [dbo].[Aircraft] TO [Airport_Staff];

GRANT SELECT, INSERT, UPDATE, DELETE ON [dbo].[Flights] TO [Airline_Internal_Staff];
GRANT SELECT, INSERT, UPDATE, DELETE ON [dbo].[Routes] TO [Airline_Internal_Staff];
GRANT SELECT, INSERT, UPDATE, DELETE ON [dbo].[Aircraft] TO [Airline_Internal_Staff];
DENY UPDATE, DELETE ON [dbo].[TicketAuditLog] TO [Airline_Internal_Staff];

-- InfoDesk
CREATE LOGIN [Narine_Info] WITH PASSWORD = 'InfoPassword123!';
CREATE USER [Narine_Info] FOR LOGIN [Maria_Info];
ALTER ROLE [InfoDesk_Agent] ADD MEMBER [Maria_Info];

-- Staff
CREATE LOGIN [Karen_CheckIn] WITH PASSWORD = 'CheckInPassword123!';
CREATE USER [Karen_CheckIn] FOR LOGIN [Oleg_CheckIn];
ALTER ROLE [Airport_Staff] ADD MEMBER [Oleg_CheckIn];

-- Manager
CREATE LOGIN [Pet_Airline] WITH PASSWORD = 'AdminPassword123!';
CREATE USER [Pet_Airline] FOR LOGIN [Boss_Airline];
ALTER ROLE [Airline_Internal_Staff] ADD MEMBER [Boss_Airline];
```

Figure 14: DCL Statement

Documentation of Views, Indexes, Triggers, and Stored Procedures

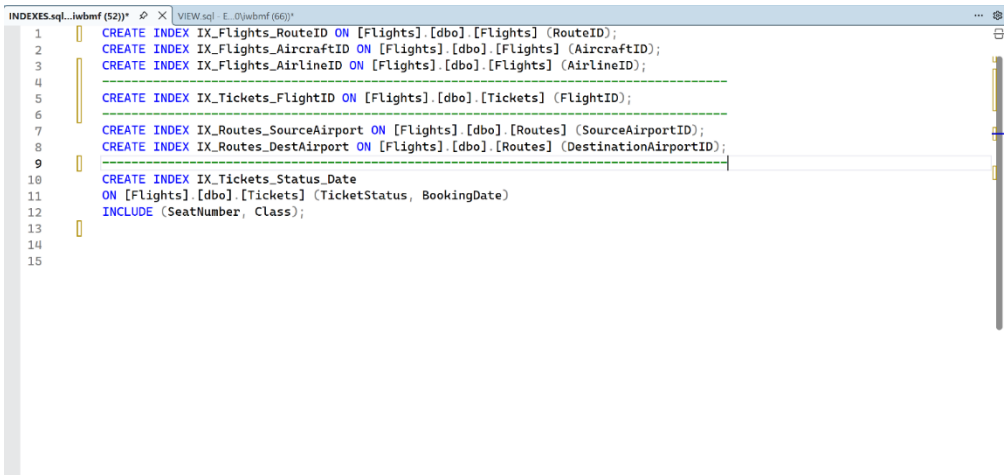
This section presents the implementation of Views, Indexes, Triggers, and Stored Procedures within the database system as part of the physical design stage. These components were developed to support efficient data management and system functionality.

The following screenshots illustrate the creation, configuration, and execution of these database objects, demonstrating their integration and correct operation within the system.



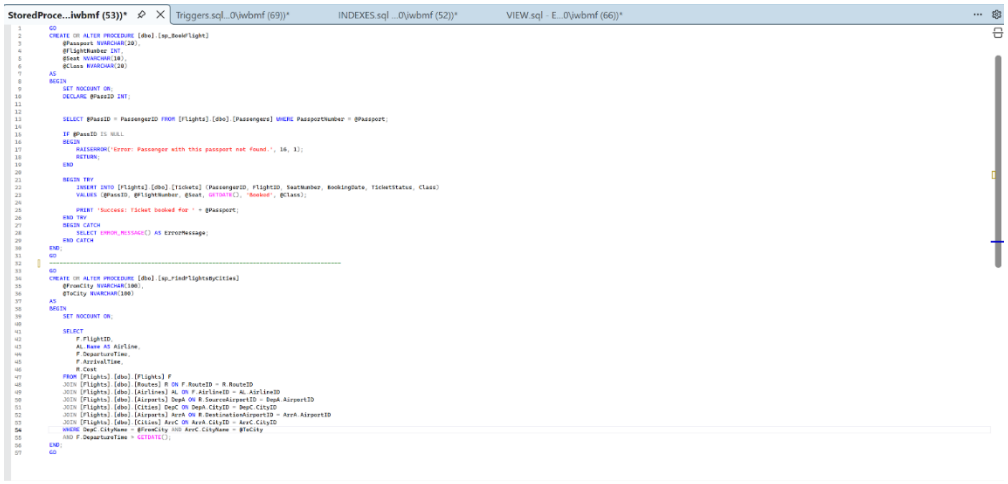
```sql
GO
CREATE OR ALTER VIEW [dbo].[View_FlightBoard] AS
SELECT
    F.FlightID,
    AL.Name AS Airline,
    A.Model AS Aircraft,
    DepAir.Code AS FromAirport,
    ArrAir.Code AS ToAirport,
    F.DepartureTime,
    F.ArrivalTime,
    R.Cost AS TicketPrice
FROM [Flights].[dbo].[Flights] F
LEFT JOIN [Flights].[dbo].[Routes] R ON F.RouteID = R.RouteID
LEFT JOIN [Flights].[dbo].[Airlines] AL ON F.AirlineID = AL.AirlineID
LEFT JOIN [Flights].[dbo].[Aircraft] A ON F.AircraftID = A.AircraftID
LEFT JOIN [Flights].[dbo].[Airports] DepAir ON R.SourceAirportID = DepAir.AirportID
LEFT JOIN [Flights].[dbo].[Airports] ArrAir ON R.DestinationAirportID = ArrAir.AirportID;
GO
-------------------------------------------------------------------------------
GO
CREATE OR ALTER VIEW [dbo].[View_RouteProfit] AS
SELECT
    R.RouteID,
    DepCity.CityName + ' - ' + ArrCity.CityName AS RouteName,
    COUNT(T.TicketID) AS TotalTicketsSold,
    SUM(CASE WHEN T.TicketStatus = 'Completed' THEN R.Cost ELSE 0 END) AS TotalRevenue
FROM [Flights].[dbo].[Routes] R
JOIN [Flights].[dbo].[Airports] DepAir ON R.SourceAirportID = DepAir.AirportID
JOIN [Flights].[dbo].[Cities] DepCity ON DepAir.CityID = DepCity.CityID
JOIN [Flights].[dbo].[Airports] ArrAir ON R.DestinationAirportID = ArrAir.AirportID
JOIN [Flights].[dbo].[Cities] ArrCity ON ArrAir.CityID = ArrCity.CityID
LEFT JOIN [Flights].[dbo].[Flights] F ON R.RouteID = F.RouteID
LEFT JOIN [Flights].[dbo].[Tickets] T ON F.FlightID = T.FlightID
GROUP BY R.RouteID, DepCity.CityName, ArrCity.CityName;
GO
-------------------------------------------------------------------------------
GO
CREATE OR ALTER VIEW [dbo].[View_PassengerManifest] AS
SELECT
    T.TicketID,
    T.FlightID,
    P.FirstName + ' ' + P.LastName AS FullName,
    P.PassportNumber,
    T.SeatNumber,
    T.Class,
    T.TicketStatus
FROM [Flights].[dbo].[Tickets] T
JOIN [Flights].[dbo].[Passengers] P ON T.PassengerID = P.PassengerID;
GO
```

Figure 15: Views



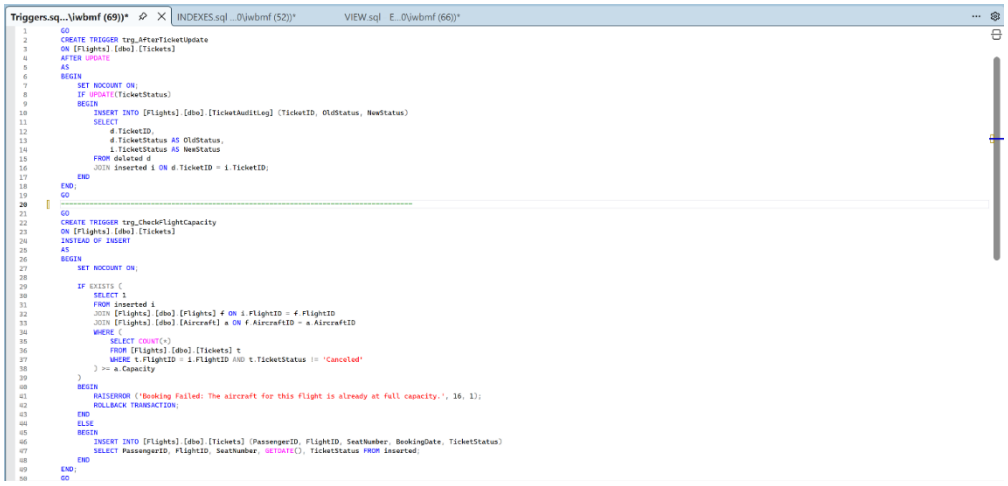
```sql
CREATE INDEX IX_Flights_RouteID ON [Flights].[dbo].[Flights] (RouteID);
CREATE INDEX IX_Flights_AircraftID ON [Flights].[dbo].[Flights] (AircraftID);
CREATE INDEX IX_Flights_AirlineID ON [Flights].[dbo].[Flights] (AirlineID);
-------------------------------------------------------------------------------
CREATE INDEX IX_Tickets_FlightID ON [Flights].[dbo].[Tickets] (FlightID);
-------------------------------------------------------------------------------
CREATE INDEX IX_Routes_SourceAirport ON [Flights].[dbo].[Routes] (SourceAirportID);
CREATE INDEX IX_Routes_DestAirport ON [Flights].[dbo].[Routes] (DestinationAirportID);
-------------------------------------------------------------------------------
CREATE INDEX IX_Tickets_Status_Date
ON [Flights].[dbo].[Tickets] (TicketStatus, BookingDate)
INCLUDE (SeatNumber, Class);
```

Figure 16 Indexes



```
GO
CREATE OR ALTER PROCEDURE [dbo].[sp_BookFlight]
    @Passport NVARCHAR(20),
    @FlightNumber INT,
    @Seat NVARCHAR(10),
    @Class NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @PassID INT;


    SELECT @PassID = PassengerID FROM [Flights].[dbo].[Passengers] WHERE PassportNumber = @Passport;

    IF @PassID IS NULL
    BEGIN
        RAISERROR('Error: Passenger with this passport not found.', 16, 1);
        RETURN;
    END

    BEGIN TRY
        INSERT INTO [Flights].[dbo].[Tickets] (PassengerID, FlightID, SeatNumber, BookingDate, TicketStatus, Class)
        VALUES (@PassID, @FlightNumber, @Seat, GETDATE(), 'Booked', @Class);

        PRINT 'Success: Ticket booked for ' + @Passport;
    END TRY
    BEGIN CATCH
        SELECT ERROR_MESSAGE() AS ErrorMessage;
    END CATCH
END;
GO
-------------------------------------------------------------------------------
GO
CREATE OR ALTER PROCEDURE [dbo].[sp_FindFlightsByCities]
    @FromCity NVARCHAR(100),
    @ToCity NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        F.FlightID,
        AL.Name AS Airline,
        F.DepartureTime,
        F.ArrivalTime,
        R.Cost
    FROM [Flights].[dbo].[Flights] F
    JOIN [Flights].[dbo].[Routes] R ON F.RouteID = R.RouteID
    JOIN [Flights].[dbo].[Airlines] AL ON F.AirlineID = AL.AirlineID
    JOIN [Flights].[dbo].[Airports] DepA ON R.SourceAirportID = DepA.AirportID
    JOIN [Flights].[dbo].[Cities] DepC ON DepA.CityID = DepC.CityID
    JOIN [Flights].[dbo].[Airports] ArrA ON R.DestinationAirportID = ArrA.AirportID
    JOIN [Flights].[dbo].[Cities] ArrC ON ArrA.CityID = ArrC.CityID
    WHERE DepC.CityName = @FromCity AND ArrC.CityName = @ToCity
    AND F.DepartureTime > GETDATE();
END;
GO
```

Figure 17: Stored Procedures



```
GO
CREATE TRIGGER trg_AfterTicketUpdate
ON [Flights].[dbo].[Tickets]
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF UPDATE(TicketStatus)
    BEGIN
        INSERT INTO [Flights].[dbo].[TicketAuditLog] (TicketID, OldStatus, NewStatus)
        SELECT
            d.TicketID,
            d.TicketStatus AS OldStatus,
            i.TicketStatus AS NewStatus
        FROM deleted d
        JOIN inserted i ON d.TicketID = i.TicketID;
    END
END;
GO
-------------------------------------------------------------------------------
GO
CREATE TRIGGER trg_CheckFlightCapacity
ON [Flights].[dbo].[Tickets]
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN [Flights].[dbo].[Flights] f ON i.FlightID = f.FlightID
        JOIN [Flights].[dbo].[Aircraft] a ON f.AircraftID = a.AircraftID
        WHERE (
            SELECT COUNT(*)
            FROM [Flights].[dbo].[Tickets] t
            WHERE t.FlightID = i.FlightID AND t.TicketStatus != 'Canceled'
        ) >= a.Capacity
    )
    BEGIN
        RAISERROR ('Booking Failed: The aircraft for this flight is already at full capacity.', 16, 1);
        ROLLBACK TRANSACTION;
    END
    ELSE
    BEGIN
        INSERT INTO [Flights].[dbo].[Tickets] (PassengerID, FlightID, SeatNumber, BookingDate, TicketStatus)
        SELECT PassengerID, FlightID, SeatNumber, GETDATE(), TicketStatus FROM inserted;
    END
END;
GO
```

Figure 18: Triggers

Database Deployment Description

The Flight Management System database was developed in SQL Server Management Studio, with TCP/IP enabled and firewall configured for remote access. It includes tables for Flights, Tickets, Routes, and Airports with proper keys. Data was populated via SQL INSERTs, and Views, Indexes, Triggers, and Stored Procedures were implemented for optimization and automation.

The front-end (JavaScript/React) interacts with the backend (Java HttpServer), sending HTTP requests that execute SQL queries and return JSON data. Users can view, add, and delete records. Data processing algorithms from the Data Structure course were used to ensure efficient search, sorting, and retrieval of flight information.

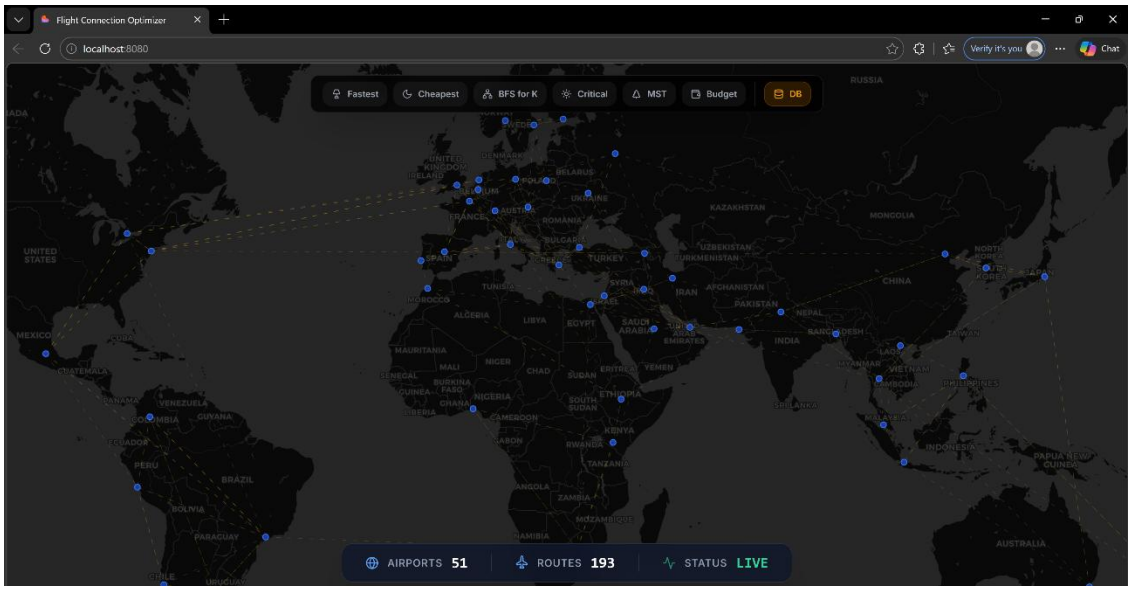
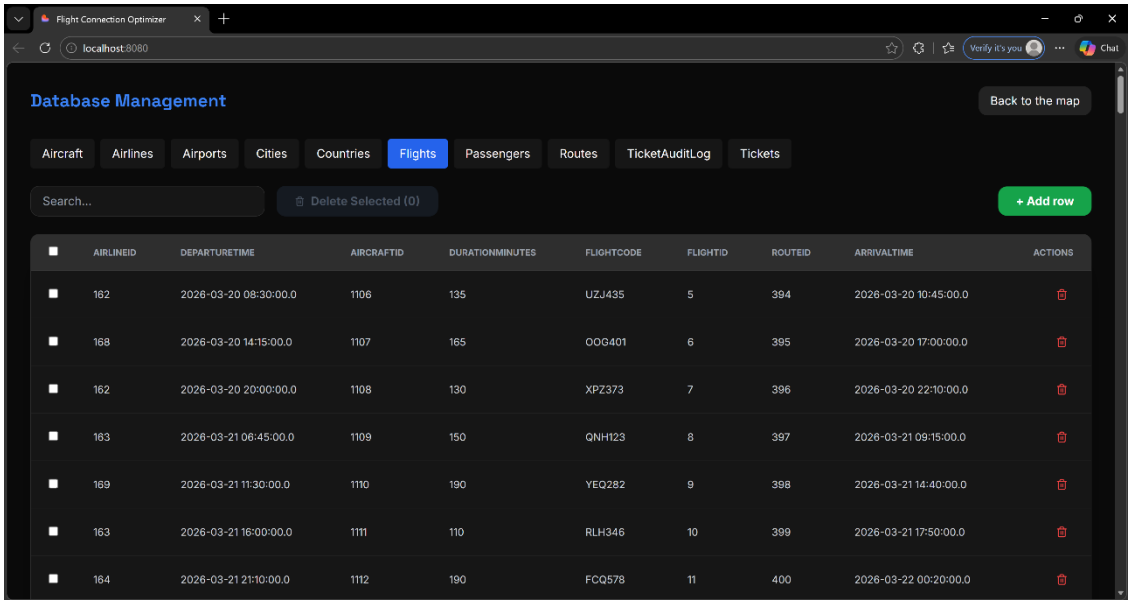


Figure 18: Main Page of Website



AIRLINEID	DEPARTURETIME	AIRCRAFTID	DURATIONMINUTES	FLIGHTCODE	FLIGHTID	ROUTEID	ARRIVALTIME	ACTIONS
162	2026-03-20 08:30:00.0	1106	135	UZJ435	5	394	2026-03-20 10:45:00.0	
168	2026-03-20 14:15:00.0	1107	165	OOG401	6	395	2026-03-20 17:00:00.0	
162	2026-03-20 20:00:00.0	1108	130	XPZ373	7	396	2026-03-20 22:10:00.0	
163	2026-03-21 06:45:00.0	1109	150	QNH123	8	397	2026-03-21 09:15:00.0	
169	2026-03-21 11:30:00.0	1110	190	YEQ282	9	398	2026-03-21 14:40:00.0	
163	2026-03-21 16:00:00.0	1111	110	RLH346	10	399	2026-03-21 17:50:00.0	
164	2026-03-21 21:10:00.0	1112	190	FCQ578	11	400	2026-03-22 00:20:00.0	

Figure 19: DB Page of Website

Project Conclusions and Results

The project successfully designed and implemented a relational database for airline operations, addressing the challenge of maintaining consistency and integrity across multiple interconnected entities. Core tables—including Aircraft, Airlines, Airports, Routes, Flights, Passengers, and Tickets—were structured with primary and foreign key constraints, ensuring correct relationships and referential integrity.

Data population, indexing, and the implementation of Views, Triggers, and Stored Procedures enabled efficient query execution, automated routine processes, and supported full CRUD operations. The database reliably enforces business rules, tracks ticketing history through audit logs, and supports analytical queries for operational insights.

Overall, the system provides a robust, consistent, and extensible solution for managing complex airline data, resolving the primary problem of ensuring data accuracy and traceability across all related entities.

GitHub link:

https://github.com/Artyomkrtchyan/FlightManagmentSystem